HARAMAYA UNIVERSITY

College of Computing and Informatics

Department of Software Engineering

Target Group:- 3rd Year Software Engineering Students

Course Name:- Requirements Engineering

Compiled By: Shambel Adugna

CHAPTER – 4

Requirement Specification

Compiled By: Shambel Adugna

Requirements specification

- The process of writing user and system requirements in a requirements document.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
  - It is therefore important that these are as complete as possible.

Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specification.

Ways of writing a system requirements specification

Notation	Description
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Requirements and design

- In principle, requirements should state what the system should do and the design should describe how it does this.
- In practice, requirements and design are inseparable
  - A system architecture may be designed to structure the requirements;
  - The system may inter-operate with other systems that generate design requirements;
  - The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
  - This may be the consequence of a regulatory requirement.

Natural language specification

- Requirements are written as natural language sentences supplemented by diagrams and tables.
- Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

Guidelines for writing requirements

- Invent a standard format and use it for all requirements.
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.

Problems with natural language

- Lack of clarity
  - Precision is difficult without making the document difficult to read.
- Requirements confusion
  - Functional and non-functional requirements tend to be mixed-up.
- Requirements amalgamation
  - Several different requirements may be expressed together.

Documentation Requirements

- General requirements of all software documentation
  - Should provide for communication among team members.
  - Should act as an information repository to be used by maintenance engineers.
  - Should provide enough information to management to allow them to perform all program management related activities.
  - Should describe to users how to operate and administer the system.

Documentation Requirements

- In all software projects some amount of documentation should be created prior to any code being written
  - Design docs, etc.
- Documentation should continue after the code has been completed
  - User's manuals, etc.
- The two main types of documentation created are ***Process*** and ***Product*** documents.

Process Documentation

- Used to record and track the development process
  - Planning documentation
  - Cost, Funding tracking
  - Schedules
  - Standards
  - Etc.
- This documentation is created to allow for successful management of a software product

Product Documentation

- Describes the **delivered** product
- Must evolve with the development of the software product
- Two main categories:
  - System Documentation
  - User Documentation

Product Documentation

- System Documentation
  - Describes how the system works, but not how to operate it
- Examples:
  - Architectural Design
  - Detailed Design
  - Commented Source Code
    - Including output such as JavaDoc
  - Test Plans
    - Including test cases
  - V&V plan and results
  - List of Known Bugs

Product Documentation

- User Documentation has two main types
  - End User
  - System Administrator
- In some cases these are the same people
  - The target audience must be well understood!

Product Documentation

- There are five important areas that should be documented for a formal release of a software application
  - These do not necessarily each have to have their own document, but the topics should be covered thoroughly

1. Functional Description of the Software
2. Installation Instructions
3. Introductory Manual
4. Reference Manual
5. System Administrator's Guide

Document Quality

- Providing thorough and professional documentation is important for any size product development team
- The problem is that many software professionals lack the writing skills to create professional level documents

Guide line to write quality SRS(document)

The general guidelines that will help to improve the quality of the document that you produce are as follows:

i. Make sure your document is well organized.

ii. Make sure that your document is well laid out.

iii. Make sure that your document is written in a style that all stakeholders will understand.

iv. All information contained in the document must be accurate and complete:

Characteristics of quality SRS

1. **Correct** :To ensure that SRS correctly reflects the actual needs.
2. **Unambiguous** :An SRS is unambiguous if, and only if, every requirement stated has only one interpretation.
3. **Complete** :All conditions under which the requirement applies are stated and it expresses a whole idea or statement
4. **Consistent** :Refers to internal consistency, and must ensure that it does not conflict with other documents
5. **Verifiable** : An SRS is verifiable if, and only if, every requirement stated .
6. **Traceable** :An SRS is traceable if the origin of each of its requirements is clear.

Contd.

7. **Modifiable**: Related concerns must be grouped together and unrelated concerns must be separated. Requirements document must have a logical structure to be modifiable.
8. **Ranked for importance and/or stability**: Each requirement in an SRS must be ranked for importance and/or stability.
9. **Valid**:To validate a requirements specification all the project participants, managers, engineers and customer representatives, must be able to understand, analyze and accept or approve it.
10. **Testable**: Requiring that a system must be “easy” to use is subjective and therefore is not testable.

Document Structure

All documents for a given product should have a similar structure.

- A good reason for *product standards*

The IEEE Standard for User Documentation lists such a structure

- It is a superset of what most documents need

The authors "best practices" are:

✓ Put a cover page on all documents

✓ Divide documents into chapters with sections and subsections

✓ Add an index if there is lots of reference information

✓ Add a glossary to define ambiguous terms

Standards

- Standards play an important role in the development, maintenance and usefulness of documentation
- Standards can act as a basis for quality documentation.
- But are not good enough on their own
    - Usually define high level content and organization

Standards

- IEEE
  - Has a published standard for user documentation
  - Provides a structure and superset of content areas
  - Many organizations probably won’t create documents that completely match the standard

Standard Templates

- Ensures that all the essential information is included
- A number of different large organizations such as the US Department of Defense and IEEE have defined standards for requirements documents
- Probably the most acceptable of these standards is the IEEE/ANSI 830-1993
- The standard recognizes differences between systems, and allows for some variations in the structure.
- There is a list of stable and variant parts:
  - Stable parts
  - Variant parts

IEEE/ANSI 830 - 1993

1. Introduction:

- Purpose of the requirements document
- Scope of product
- Definitions, acronyms and abbreviations
- References
- Overview of the remainder of the document

2. General Description:

- Product perspective
- Product functions
- User characteristics
- General constraints
- Assumptions and dependencies

IEEE/ANSI 830 - 1993

3. Specific Requirements

- Covering functional, non functional & interface requirements, these should document
  - External interfaces
  - Functionality
  - Performance requirement,
  - Logical database requirement,
  - Design constraints,
  - System attributes
  - Quality characteristics

IEEE/ANSI 830 - 1993

4. Appendices
5. index

1.0 Introduction
- 1.1 Purposes
- 1.2 Scope
- 1.3 Definitions, Acronyms, and Abbreviations
- 1.4 References
- 1.5 Overview

2.0 The Overall Description
- 2.1 Product Perspective
  - 2.1.1 System Interface
  - 2.1.2 Interface
  - 2.1.3 Hardware Interface
  - 2.1.4 Software Interface
  - 2.1.5 Communications Interface
  - 2.1.6 Memory Constraints
  - 2.1.7 Operations
  - 2.1.8 Site Adaptation Requirements
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions and Dependency
- 2.6 Apportioning of Requirements

3.0 Specific Requirements
- 3.1 External Interface
- 3.2 Functions
- 3.3 Performance Requirements
- 3.4 Logical Database of Requirement
- 3.5 Design Constraints
  - 3.5.1 Standards Compliance
- 3.6 Software System Attributes
  - 3.6.1 Reliability
  - 3.6.2 Availability
  - 3.6.3 Security
  - 3.6.4 Maintainability
  - 3.6.5 Portability
- 3.7 Organizing the Specific Requirements
  - 3.7.1 System Mode
  - 3.7.2 User Class
  - 3.7.3 Objects
  - 3.7.4 Feature
  - 3.7.5 Stimulus
  - 3.7.6 Response
  - 3.7.7 Functional Hierarchy
- 3.8 Additional Comments

4.0 Change Management Process

5.0 Document Approvals

6.0 Supporting Information

System modelling

- **System modeling** is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modelling helps the
  - ***Analyst to understand the functionality of the system***
  - ***Models are used to communicate with customers.***

System modelling…

- **System modeling** has generally come to mean representing the system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).
- However, it is also possible to develop formal (mathematical) models of a system, usually as a detailed system specification.

System modelling…

- Models are used during the requirements engineering process to help derive **the requirements for a system,** during the **design process to** describe the **system to engineers implementing the system** and after implementation to document the system's structure and operation.

System modelling…

- You may develop models of **both the existing system** and **the system to be developed**:

  1. **Models of the existing system are used during requirements engineering.**

  2. **Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders.**

Software modelling and models

- **Software modelling** helps the engineer to understand the functionality of the system
- **Models** are used for communication among stakeholders
- **Different models present** the system from different perspectives
  - **External perspective** showing the system's context or environment
  - **Process models** showing the system development process as well as activities supported by the system
  - **Behavioural perspective** showing the behaviour of the system
  - **Structural perspective** showing the system or data architecture

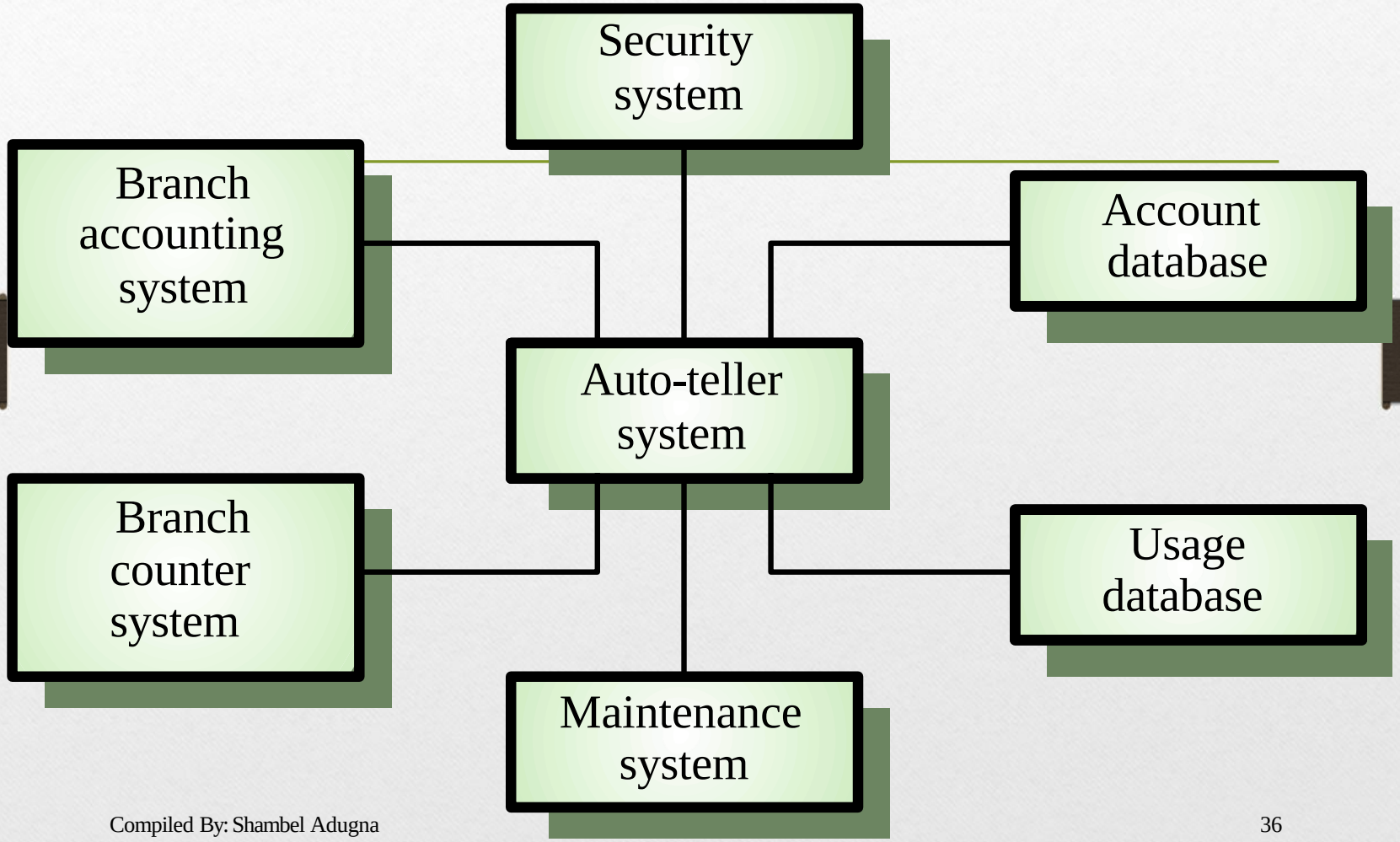
Model types

- **Data processing model** showing how the data is processed at different stages
- **Composition model** showing how entities are composed of other entities
- **Architectural model** showing principal sub-systems
- **Classification model** showing how entities have common characteristics
- **Stimulus/response model** showing the system's reaction to events

1. Context models

- **Context models** are used to illustrate the boundaries of a system
- Social ~~and organisational concerns may affect the decision on~~ where to position system boundaries
- **Architectural models** show the a system and its relationship with other systems

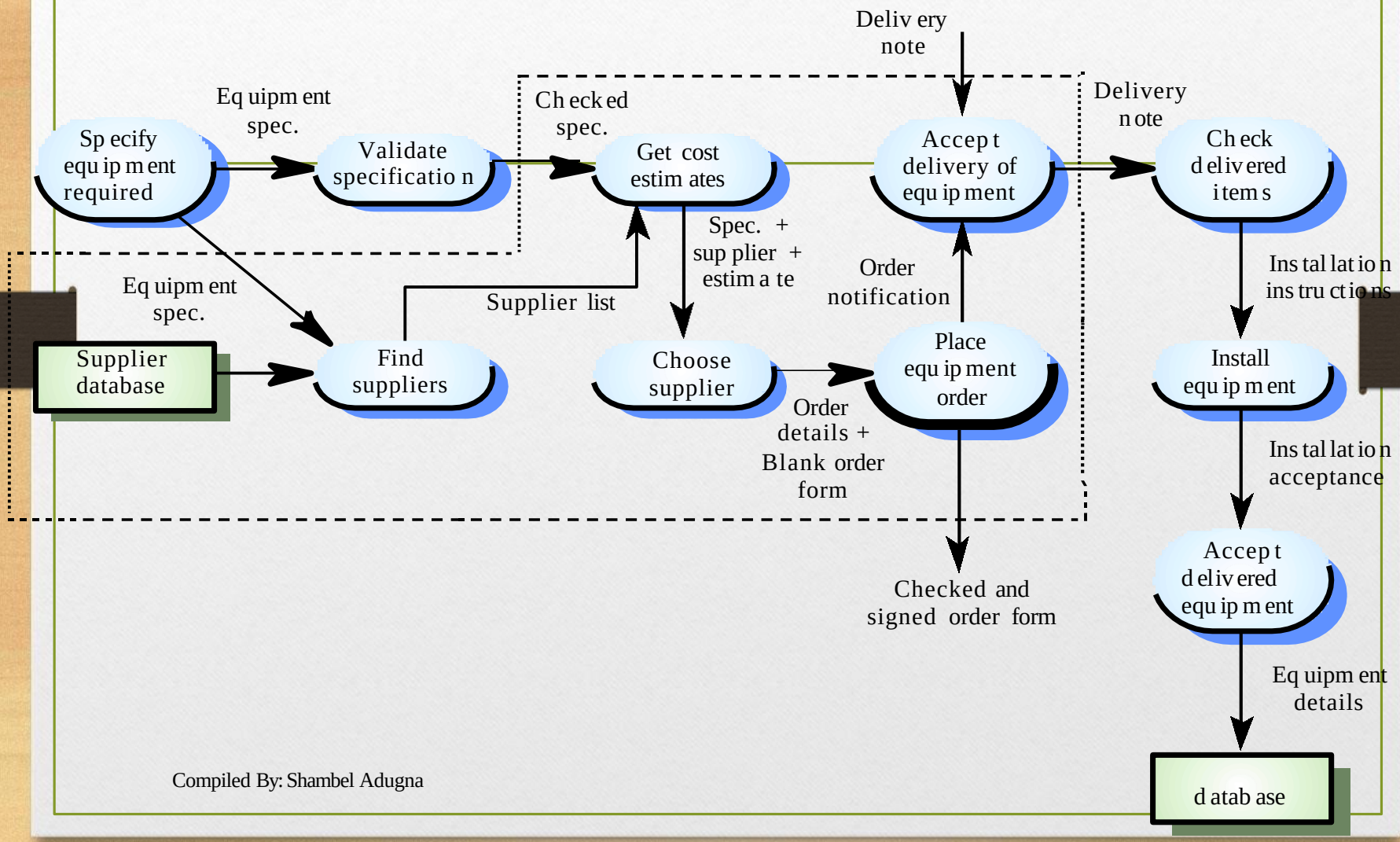
The context of an ATM system



Process models

- **Process models** show the overall process and the processes that are supported by the system
- **Data flow models** may be used to show the processes and the flow of information from one process to another

Equipment obtaining process



Compiled By: Shambel Adugna

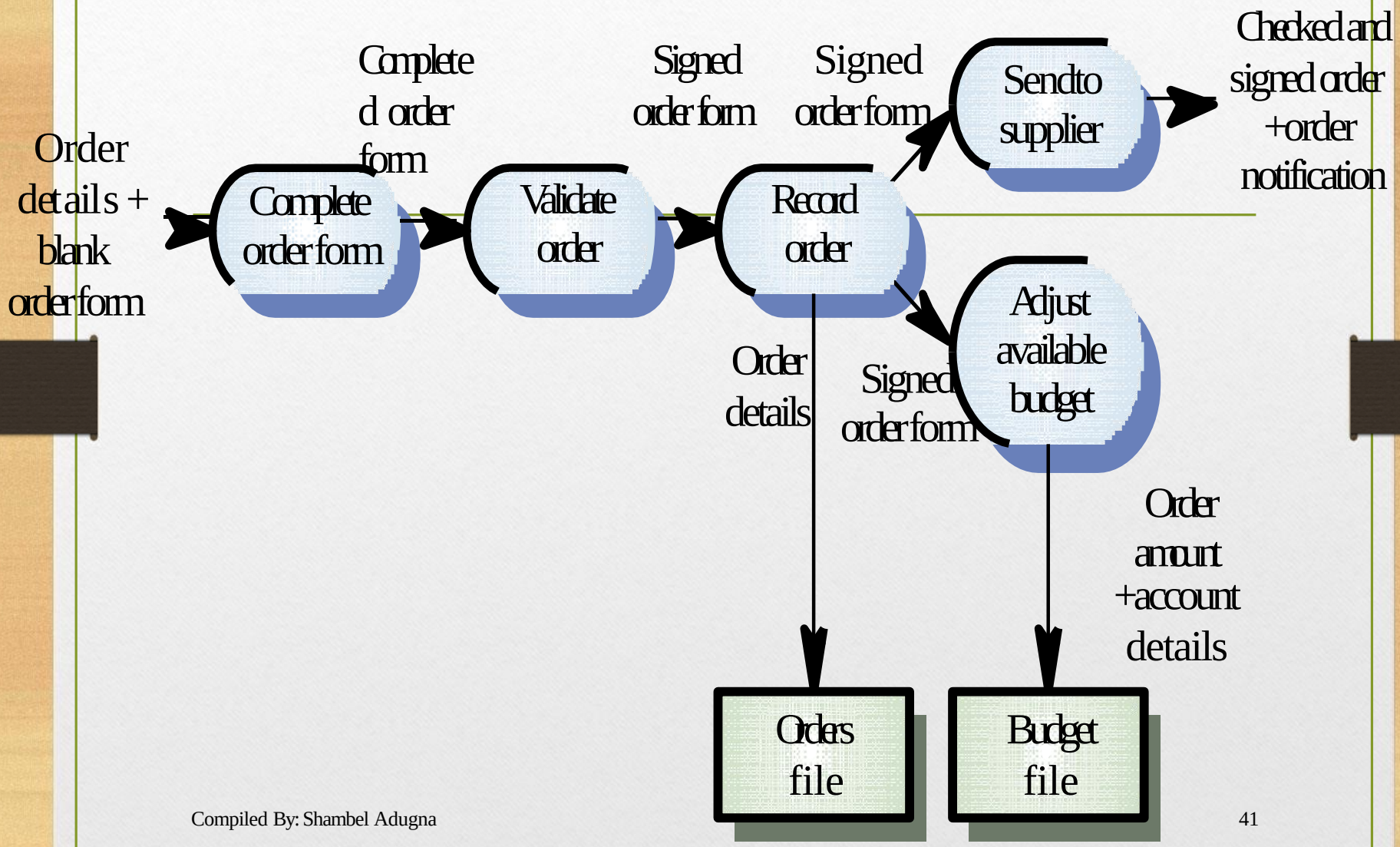
2. Behavioural models

- Behavioural models are used to describe the overall behaviour of a system
- Two types of behavioural model
  - **Data processing models** that show how data is processed as it moves through the system
  - **State machine models** that show the systems response to events
- Both of these models are required for a description of the system's behaviour

2.1 Data-processing models

- Data flow diagrams are used to model the system's data processing
- These show the processing steps as data flows through a system
- IMPORTANT part of many analysis methods
  - Simple and intuitive notation that customers can understand
  - Show end-to-end processing of data

Order processing DFD



Data flow diagrams

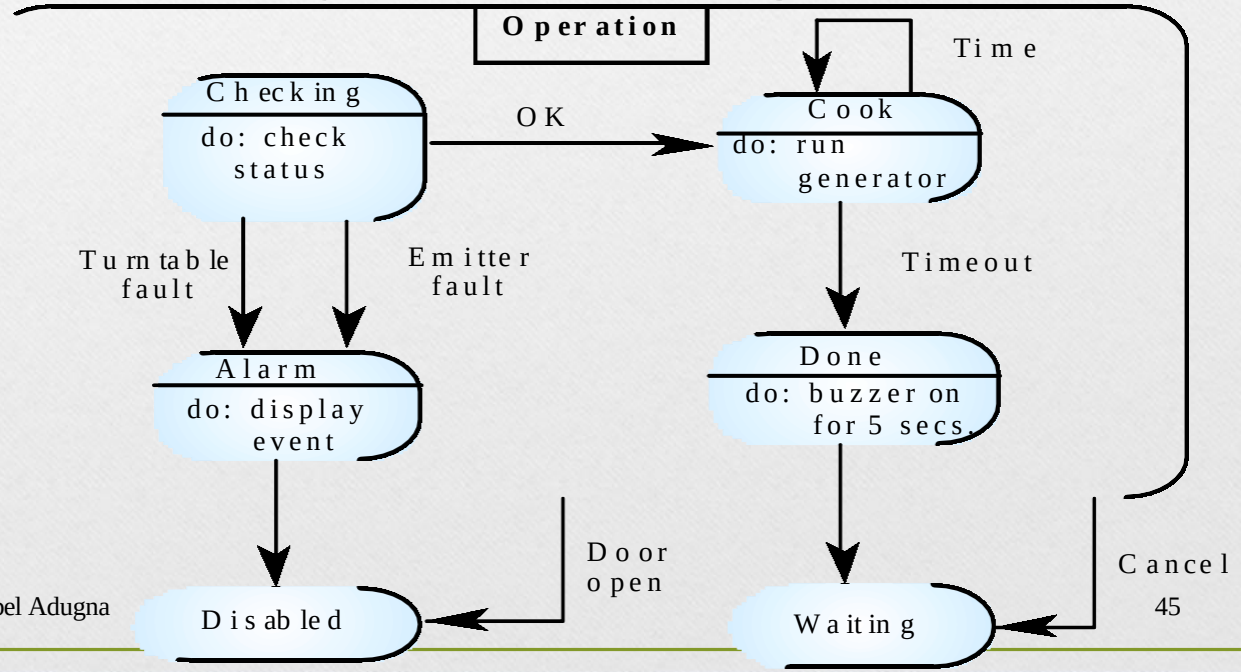
- DFDs model the system from a functional perspective
- Tracking and documenting how the data associated with a process is helpful to develop an overall understanding of the system
- Data flow diagrams may also be used in showing the data exchange between a system and other systems in its environment

2.2 State machine models

- State Machine models the behaviour of the system in response to external and internal events
- They ~~show the system's responses to stimuli so are often us~~ed for modelling real-time systems
- State machine models show system states as nodes and events as arcs between these nodes. When an event occurs, the system moves from one state to another
- **State charts** are an integral part of the UML

State charts

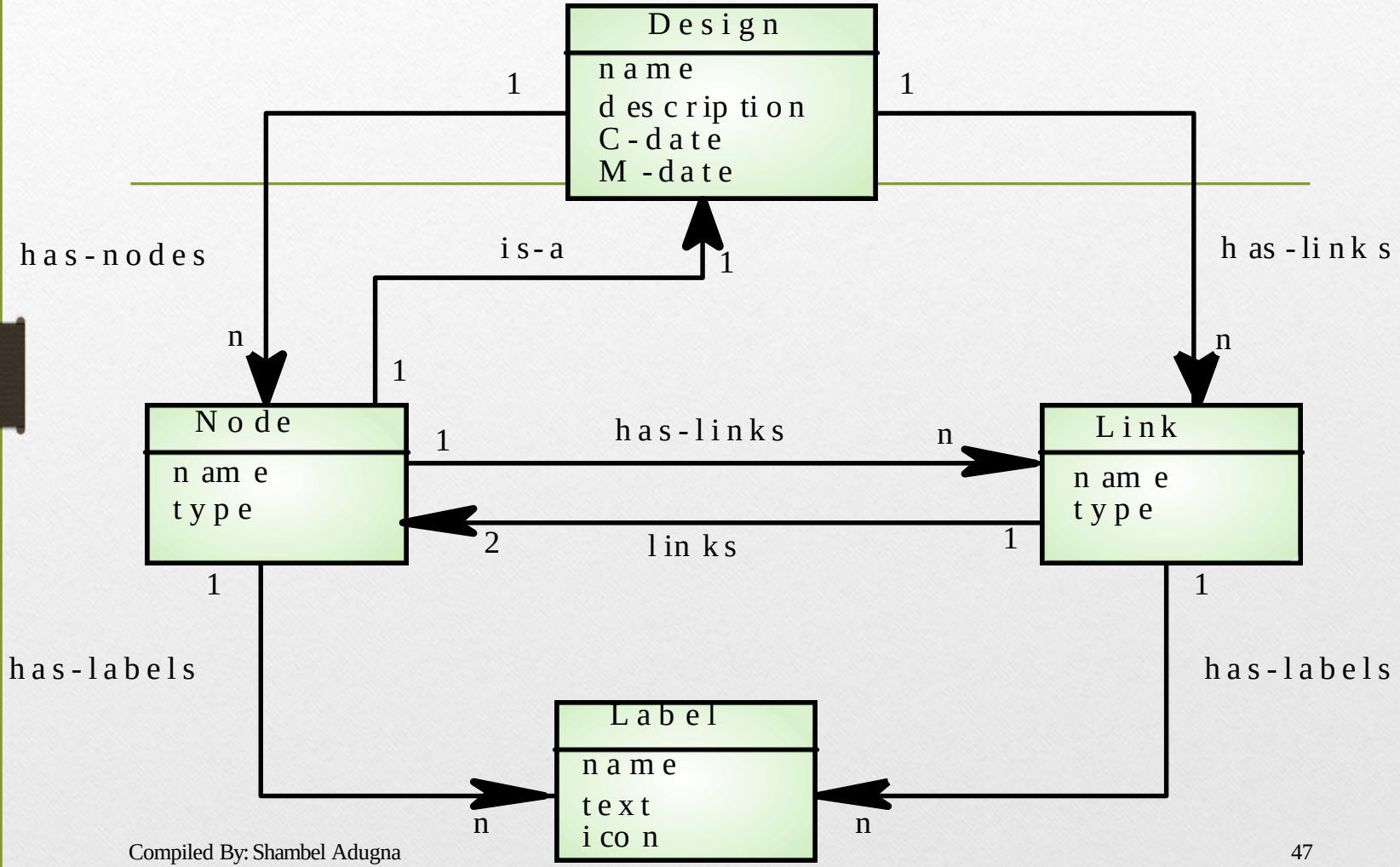
- Allow the decomposition of a model into sub-models (see a figure below)
- A brief description of the actions is included following the 'do' in each state
- Can be complemented by tables describing the states and the stimuli



3. Semantic data models

- Used to describe the logical structure of data processed by the system
- Entity-relation-attribute model sets out the entities in the system,
  the relationships between these entities and the entity attributes
- **Widely used in database design**. Can readily be implemented using relational databases
- No specific notation provided in the UML but objects and associations can be used

Software design semantic model



Data dictionary entries

- Data dictionaries are lists of all of the names used in the system models.
- Descriptions of the entities, relationships and attributes are also included

Name	Description	Type	Date
has-labels	1:N relation between entities of type Node or Link and entities of type Label.	Relation	5.10.1998
Label	Holds structured or unstructured information about nodes or links. Labels are represented by an icon (which can be a transparent box) and associated text.	Entity	8.12.1998
Link	A 1:1 relation between design entities represented as nodes. Links are typed and may be named.	Relation	8.12.1998
name (label)	Each label has a name which identifies the type of label. The name must be unique within the set of label types used in a design.	Attribute	8.12.1998
name (node)	Each node has a name which must be unique within a design. The name may be up to 64 characters long.	Attribute	15.11.1998

4. Object models

- Object models describe the system in terms of object classes
- An object class is an abstraction over a set of objects with common attributes and the services (operations) provided by each object
- Various object models may be produced
  - Inheritance models
  - Aggregation models
  - Interaction models

Object models

- Natural ways of reflecting the real-world entities manipulated by the system
- More abstract entities are more difficult to model using this approach
- Object class identification is recognised as a difficult process requiring a deep understanding of the application domain
- Object classes reflecting domain entities are reusable across systems

The Unified Modelling Language

- ***Devised by the developers of widely used object-oriented analysis and design methods***
- Has become an effective standard for object-oriented modelling
- Notation
  - Object classes are **rectangles** with the name at the top, attributes in the middle section and operations in the bottom section
  - Relationships between object classes (known as associations) are shown as lines **linking** objects
  - Inheritance is referred to as generalisation and is shown '**upwards**' rather than 'downwards' in a hierarchy.

END OF CHAPTER - 4